

QA evidence — RECALL + TELEMETRY security probes

Revision: 1 **Last modified:** 2026-06-10T11:40:36Z **Run-id:** 20260610T114036Z-recall-sec-probes **Probe script:** tests/security/recall_telemetry_probes.sh **Companion doc:** docs/scripts/recall_telemetry_probes.md

What this proves

Black-box, anti-bluff security coverage (Constitution §11.4 / §11.4.27 / §11.4.123) for two surfaces NOT covered by security_probes.sh / security_probes_filters.sh:

- `POST /api/v1/deployments/{id}/recall` — AUTHZ + input VALIDATION + trust boundary.
- `GET /api/v1/devices/{id}/telemetry` — resource OWNERSHIP + filter input VALIDATION.

Every probe asserts the EXACT HTTP status from a real response against a live ota-server. Status codes were read from server source (§11.4.6 no-guessing): both handlers use `respondValidation()` → **HTTP 400** (not 422).

Probe → expected-status table (the conductor verifies these live)

Probe	Request	Expected
R1	unauth <code>POST /deployments/{id}/recall</code>	401
R2	device-role token <code>POST .../recall</code>	403
R2b	device token <code>GET /client/update</code> (token-valid control)	200/204
R3	operator/admin token <code>POST .../recall</code> (valid, real fixture)	201 (SKIP w/o signed fixture)
V1	<code>POST .../recall</code> missing <code>to_release_id</code>	400 (SKIP w/o fixture)
V2	<code>POST .../recall</code> unknown <code>to_release_id</code>	404 (SKIP w/o fixture)
V3	<code>POST .../recall</code> unknown deployment id (operator)	404
O1		403

Probe	Request	Expected
	device A token GET /devices/{B}/telemetry	
O2	device A token GET /devices/{A}/telemetry (own)	200
O3	privileged token GET /devices/{B}/telemetry	200
F1	?event=<unknown>	400
F2	?since=<not-RFC3339>	400
F3a/b/c	?limit=0 / =999 / =-1	400 each
F4	?cursor=<malformed>	400
F5	? event=success&limit=10&cursor=0 (valid control)	200
T1	recall with injected public_key/ signature fields	201 (fixture) / 404 (no fixture) — fields IGNORED
*b	edge/injection inputs did not 5xx	no 5xx

Conductor run command (capture PASS evidence here)

Run against the clean merged tree (server source is final), capturing the evidence file into this directory:

```
cd /Volumes/T7/Projects/helix_ota
HELIX_SECURITY_EVIDENCE="docs/qa/20260610T114036Z-recall-sec-
    probes/RUN_EVIDENCE_RECALL_TELEMETRY.txt" \
    bash tests/security/recall_telemetry_probes.sh
echo "exit=$?" # expect 0 = PASS
```

Optional pinned port (avoids any ephemeral-port collision in CI):

```
HELIX_PORT=28765 \
HELIX_SECURITY_EVIDENCE="docs/qa/20260610T114036Z-recall-sec-
    probes/RUN_EVIDENCE_RECALL_TELEMETRY.txt" \
    bash tests/security/recall_telemetry_probes.sh --port 28765
```

For full V1/V2 recall-validation coverage, ensure openssl (≥ 3 , ed25519), xxd, base64, python3 are on PATH on the runner; otherwise V1/V2 SKIP-with-reason (honest, not a false PASS).

Status of this run

- **Parse-only** validation done by the authoring subagent (parallel phase; server source may be mid-edit by another agent, so the live probes were NOT run

here): `bash -n clean`, `sh -n clean` (§11.4.67); JSON-body substitution and per-PID port derivation dry-run-verified.

- **Live PASS evidence** is to be captured by the CONDUCTOR against the clean merged tree, written to `RUN_EVIDENCE_RECALL_TELEMETRY.txt` in this directory.